TP Final

Plataforma de datos para planificación y reposición en una cadena de indumentaria

Trabajo Práctico Final — Variante 1: Investigación (caso de negocio y arquitectura de datos) Estudiante: Gonza Tajada — Especialización en Ciencia de Datos, ITBA

> **Nota sobre el caso.** La organización es **hipotética**: una marca argentina de indumentaria con locales propios, tienda online y venta mayorista, inspirada en el tipo de operación que conozco por mi trabajo en planificación y compras. Ninguna cifra proviene de datos reales de una empresa: todo valor numérico es una **estimación** y está marcado como tal. Los supuestos están en la sección 2.3.

---

1. Necesidad de negocio

1.1 Contexto

La empresa vende indumentaria con fuerte estacionalidad (temporadas verano/invierno) a través de tres canales: **40 locales físicos** en distintas provincias, **una tienda online** (Shopify) y **clientes mayoristas**. El producto es de ciclo corto: lo que no se vende en la temporada se liquida con descuento y destruye margen.

1.2 Áreas y usuarios

- **Planificación y compras:** decide cuánto comprar y cómo distribuir entre locales.
- **Comercial / operaciones de locales:** pide y recibe mercadería, ejecuta promociones.
- **Dirección y finanzas:** miran margen, rotación e inventario inmovilizado.

1.3 Problema actual

1. Los datos viven en sistemas separados (ERP/punto de venta, Shopify, planillas de promos y precios) y se integran a mano en Excel.
2. La reposición se decide con reportes que llegan con días de atraso y sin una definición única de “venta”, “stock” o “sell-through”.
3. Resultado: **quiebres** en los talles y artículos que más rotan, y **sobrestock** en los que no. En el gráfico 1 se muestra la distribución supuesta de cobertura de stock.
4. Armar el reporte semanal consume mucho tiempo analítico (estimado: ~16 horas por semana), que no se dedica a analizar.

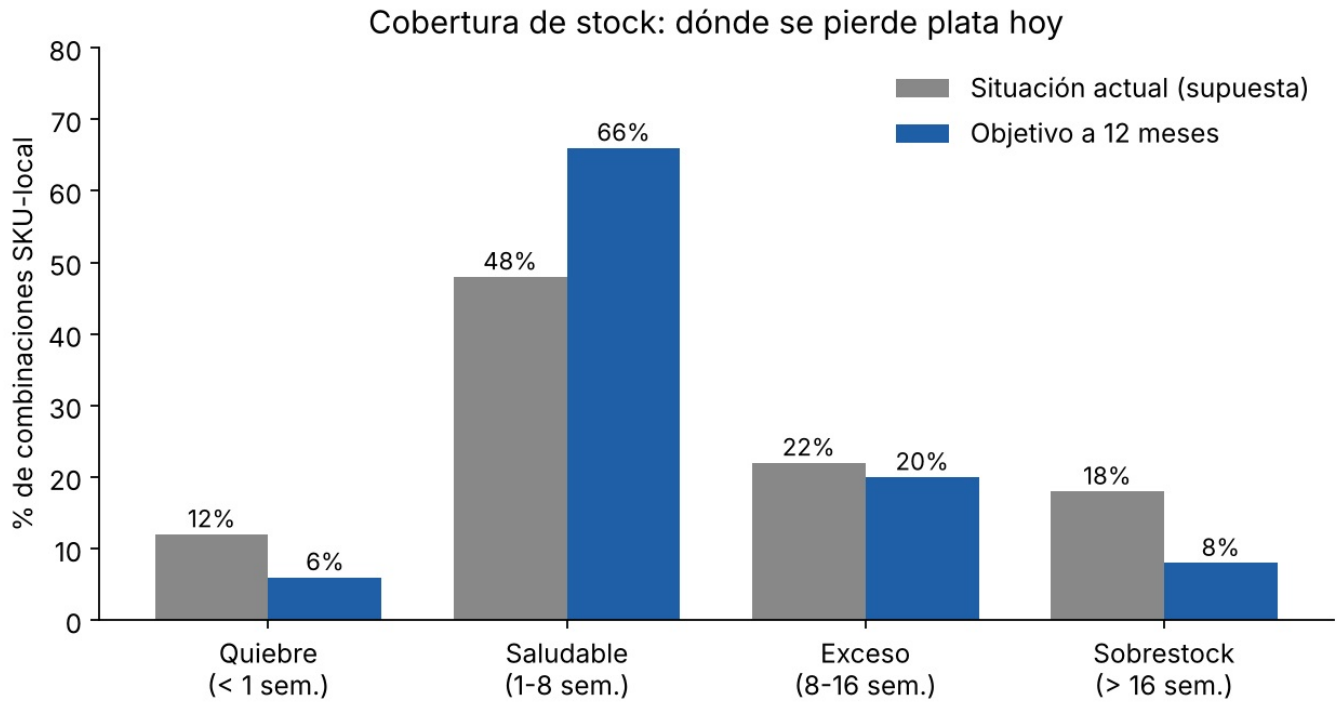


Gráfico 1. Distribución supuesta de la cobertura de stock por SKU-local, hoy vs. objetivo. Estimación.

1.4 Objetivo y beneficios esperados

Objetivo: construir una plataforma de datos única y confiable que alimente el seguimiento de ventas e inventario y una **sugerencia semi-automática de reposición** por SKU y local.

Beneficios esperados: menos ventas perdidas por quiebre, menos capital inmovilizado en sobrestock, una sola versión de la verdad para todas las áreas y tiempo analítico liberado.

1.5 Cómo se evaluaría el éxito

Se propone un **piloto de 3 meses con 8 locales** comparados contra 8 locales de control de perfil similar.

KPI	Línea base (supuesta)	Meta	Cómo se mide
Quiebres de stock (% SKU-local con cobertura < 1 semana)	12 %	6 %	`fact_stock_diario`
Sell-through a 12 semanas	55 %	62 %	`fact_ventas` / unidades recibidas
Tiempo de armado del reporte semanal	16 h	1 h	Registro del equipo
Frescura del dato en el tablero	2 a 5 días	≤ 24 h	Timestamp de última carga

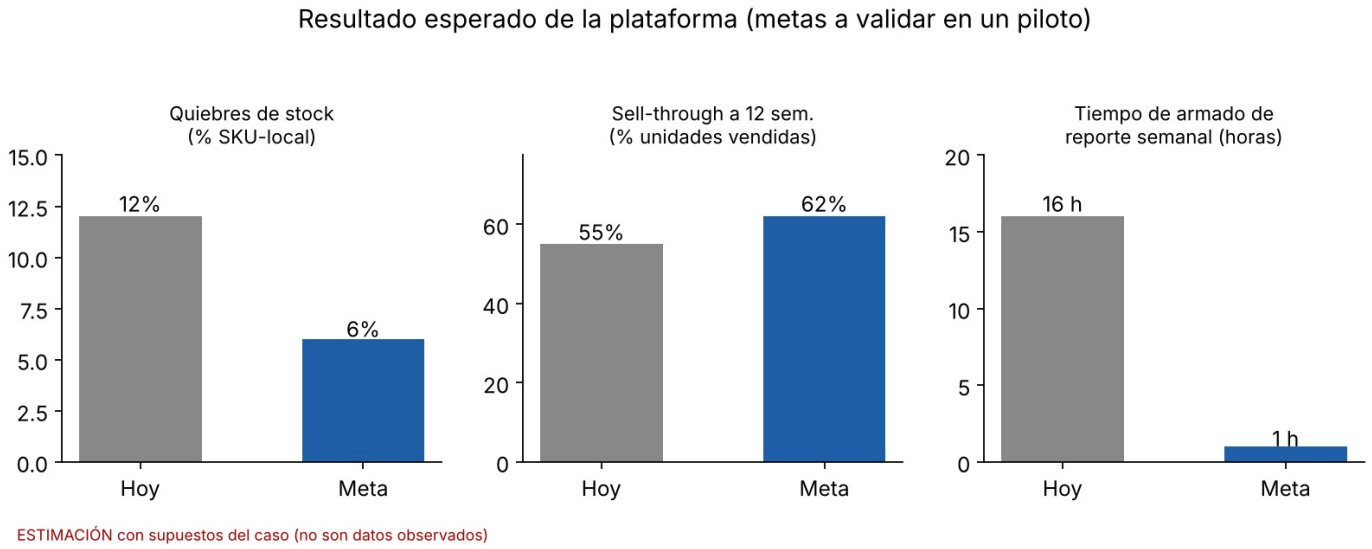


Gráfico 2. Metas del piloto. Son hipótesis a validar, no resultados.

El piloto se considera exitoso si al menos tres de los cuatro KPIs alcanzan la meta en los locales piloto y no en los de control.

---

2. Datos

2.1 Fuentes

Fuente	Contenido	Tipo	Frecuencia	Volumen estimado	Calidad esperada	Sensibilidad
POS / ERP	Líneas de ticket de locales	Estructurado (tablas/CSV)	Diaria (lote nocturno)	~3,7 M líneas/año	Media: devoluciones, anulaciones y cambios mal codificados	Media (puede incluir DNI/email del cliente)
POS / ERP	Stock por SKU y local	Estructurado	Diaria (foto de cierre)	~192.000 filas/día (~70 M/año)	Media: ajustes de inventario tardíos	Baja
Shopify	Pedidos online	Semi-estructurado (JSON por webhook/API)	Casi en tiempo real	~0,5 M líneas/año	Alta, pero con pedidos cancelados y reembolsos	Alta (datos personales de compradores)
Catálogo	SKU, categoría, temporada, talle, color, costo, precio	Estructurado	Semanal	~12.000 SKU	Media: atributos faltantes o inconsistentes entre sistemas	Media (el costo es confidencial)
Promociones y precios	Vigencias y descuentos	Planillas	Semanal	Pocos miles de filas	Baja: carga manual	Baja
Externas	IPC, tipo de cambio, feriados	API / CSV públicos	Mensual / diaria	Mínimo	Alta	Ninguna

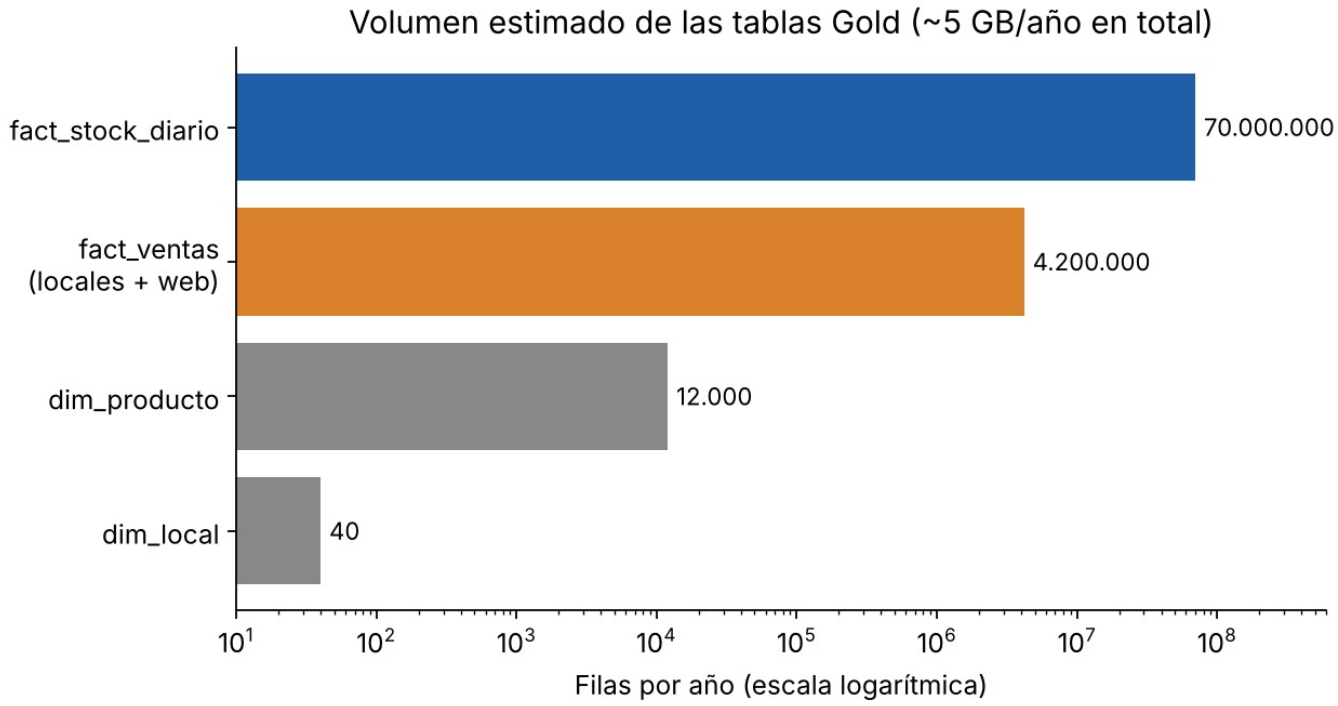
2.2 Restricciones principales

- **Ley 25.326 de Protección de Datos Personales:** los datos de clientes no se necesitan para el caso de uso de reposición, por lo que se **excluyen o seudonimizan en Silver** y no llegan a Gold.

- Los costos y márgenes son información comercial confidencial: acceso restringido por rol.
- Calidad de origen heterogénea y carga manual de promociones.
- Conectividad y horarios de cierre distintos entre locales.

2.3 Datos disponibles vs. supuestos

- **Disponible (se asume que existe en la organización):** ventas por ticket, stock por local, catálogo y pedidos online.
- **Supuestos que hago para dimensionar:** 40 locales, ~12.000 SKU (modelo-talle-color), ~40 % de las combinaciones SKU-local activas por día, ~11.500 líneas de venta por día entre todos los canales, 1 sola tienda online. Las metas de KPI son hipótesis.



ESTIMACIÓN con supuestos del caso (no son datos observados)

Gráfico 3. Volumen estimado por tabla. Estimación.

Observación honesta: ~5 GB por año **no es "Big Data" por volumen**. La justificación de la plataforma está en la **variedad** (varias fuentes y formatos), la **necesidad de historia y reproducibilidad**, el gobierno de datos y la posibilidad de crecer (más locales, más canales, sensores de tráfico). Esto se retoma en la sección 5.

---

3. Arquitectura propuesta

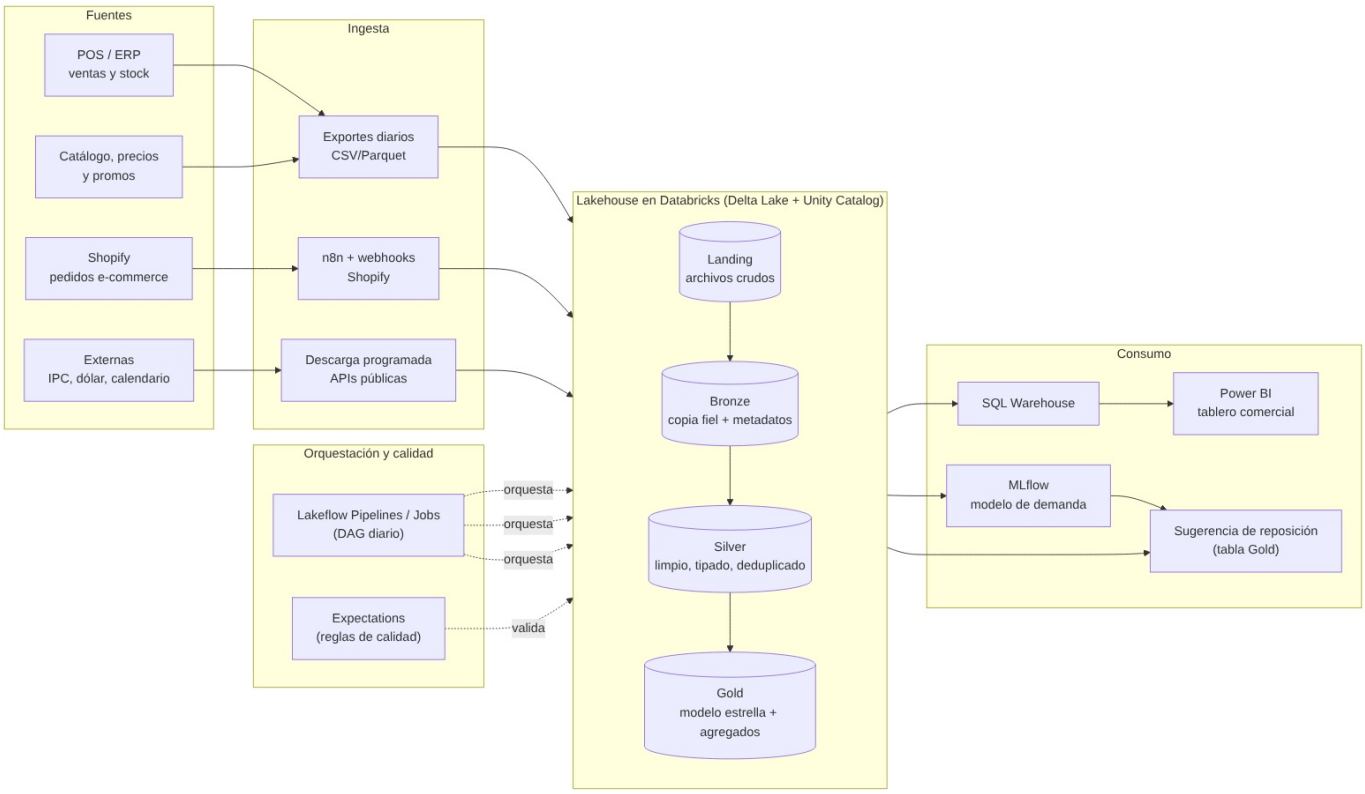


Diagrama 1. Arquitectura de datos, de la ingesta al consumo.

3.1 Recorrido de los datos

1. **Ingesta.** El POS/ERP deja **exportes diarios en Parquet/CSV** en almacenamiento en la nube (zona *landing*). Shopify envía eventos por **webhooks que recibe n8n** y los escribe como JSON en la misma zona. Las fuentes externas se descargan con tareas programadas.
2. **Bronze. Auto Loader** de Databricks lee solo los archivos nuevos y los guarda en tablas Delta sin modificar, agregando metadatos (archivo de origen, fecha de carga). Permite reprocesar.
3. **Silver.** Limpieza, tipado, **deduplicación**, tratamiento de anulaciones y devoluciones, estandarización de SKU y locales, seudonimización de datos personales. Acá se aplican las **reglas de calidad**.
4. **Gold.** Modelo estrella (sección 4) y agregados: `gold_sellthrough_semanal` y `gold_sugerencia_reposicion`.
5. **Consumo.** Un **SQL Warehouse** sirve el Gold a **Power BI** (tablero comercial) y a la sugerencia de reposición, que el equipo de compras revisa y aprueba.

3.2 Tecnologías por componente

Componente	Tecnología	Rol
Almacenamiento	Delta Lake sobre object storage (S3/ADLS)	Tablas transaccionales, versionado, *time travel*
Procesamiento	Apache Spark en Databricks (PySpark y SQL)	Transformaciones Bronze→Silver→Gold
Ingesta	Auto Loader + n8n (webhooks de Shopify)	Carga incremental de archivos y eventos
Orquestación	Pipelines/Jobs de Databricks (Lakeflow)	DAG diario con dependencias y reintentos
Calidad	*Expectations* del pipeline + tabla de cuarentena	Reglas declarativas y registro de filas rechazadas
Gobierno	Unity Catalog	Catálogo, permisos por rol, linaje, enmascarado de columnas
Visualización	Power BI sobre SQL Warehouse	Tablero para compras, comercial y dirección

3.3 Orden de ejecución (DAG diario, ~03:00)

`ingesta_pos` ‖ `ingesta_shopify` ‖ `ingesta_externas` → `bronze` → `silver` (con validación de calidad) → `gold_dimensiones` → `gold_hechos` → `gold_agregados` → `refresco_power_bi`. Si la validación de calidad falla por encima de un umbral, el pipeline se detiene y notifica en lugar de publicar datos dudosos.

---

4. Modelado de datos

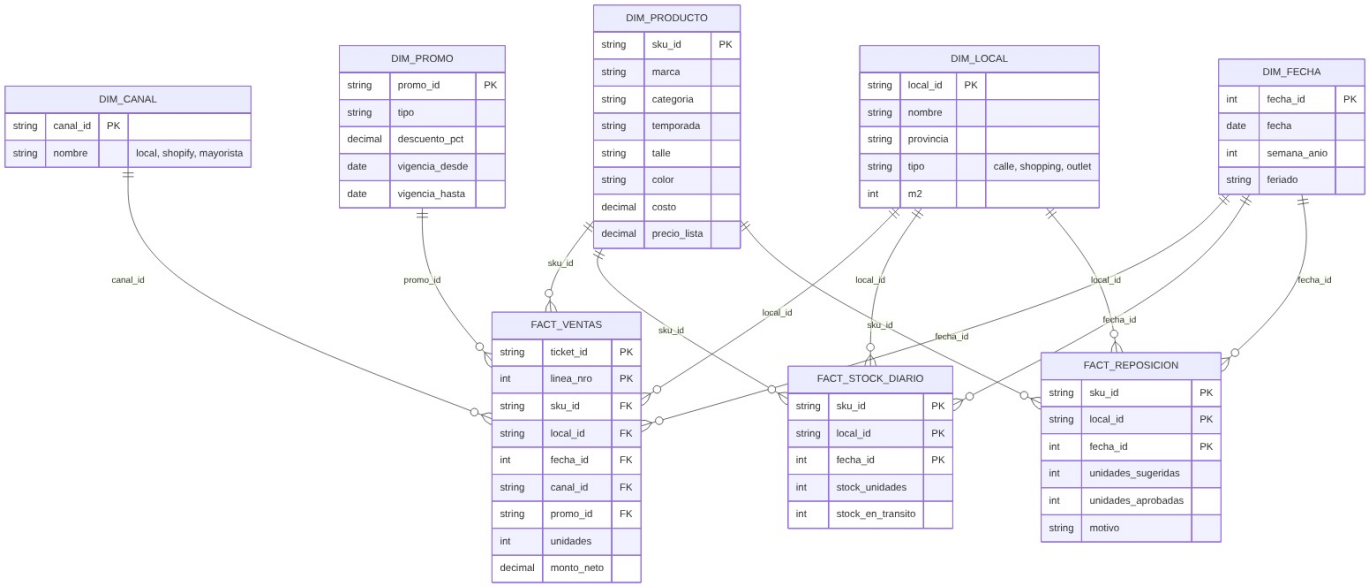


Diagrama 2. Modelo estrella de la capa Gold.

4.1 Entidades, claves y granularidad

Tabla	Tipo	Granularidad (una fila por…)	Clave
`fact_ventas`	Hecho	línea de ticket	(`ticket_id`, `linea_nro`)
`fact_stock_diario`	Hecho (foto periódica)	SKU × local × día	(`sku_id`, `local_id`, `fecha_id`)
`fact_reposicion`	Hecho	SKU × local × día de sugerencia	(`sku_id`, `local_id`, `fecha_id`)
`dim_producto`	Dimensión	SKU (modelo-talle-color)	`sku_id`
`dim_local`	Dimensión	local	`local_id`
`dim_fecha`	Dimensión	día	`fecha_id`
`dim_canal`	Dimensión	canal (local, Shopify, mayorista)	`canal_id`
`dim_promo`	Dimensión	promoción	`promo_id`

Capas anteriores: Bronze tiene una tabla por fuente (`bronze_pos_ventas`, `bronze_pos_stock`, `bronze_shopify_pedidos`, `bronze_catalogo`, `bronze_promos`) y Silver sus equivalentes limpios (`silver_ventas`, `silver_stock`, etc.) más `silver_cuarentena` con las filas rechazadas y el motivo.

4.2 Decisiones de modelado

- **Esquema en estrella** porque el consumo principal es analítico (agregar por producto, local y tiempo) y Power BI lo aprovecha bien.
- **Grano fino en ventas** (línea de ticket) para poder reagregar a cualquier nivel sin rehacer la carga.
- **Foto diaria de stock** como hecho aparte: el stock no se suma en el tiempo y se necesita para calcular cobertura y quiebres.
- **Claves sustitutas estables** para SKU y local, porque los códigos de origen difieren entre POS y Shopify.
- `fact_stock_diario` se **particiona por fecha** y se compacta/ordena por `local_id` y `sku_id` para consultas por local.

4.3 Cómo el modelo responde las preguntas de negocio

Pregunta	Cómo se responde
¿Qué SKU están en quiebre en cada local?	`fact_stock_diario` con `stock_unidades = 0` y ventas recientes en `fact_ventas`
¿Cuál es el sell-through por categoría y temporada?	`fact_ventas` ÷ unidades recibidas, por `dim_producto` y `dim_fecha`
¿Cuánto cubre el stock actual en semanas de venta?	`stock_unidades` ÷ promedio de ventas semanales de las últimas N semanas
¿Cuánto reponer a cada local?	`gold_sugerencia_reposicion`: demanda esperada − stock − tránsito, ajustada por redondeo a talles
¿Qué efecto tuvo una promo?	`fact_ventas` cruzada con `dim_promo` y comparación contra semanas sin promo

---

5. Justificación de las decisiones

Decisión	Por qué	Compromisos y limitaciones
Lakehouse (Delta Lake) y no solo un data warehouse	Mezcla datos tabulares (POS) y JSON (Shopify); Delta da transacciones ACID, historia y reprocesos sobre almacenamiento barato; además deja los datos listos para ML.	Con ~5 GB/año es más de lo estrictamente necesario: una base PostgreSQL o un warehouse administrado alcanzarían en volumen. Se elige lakehouse por **variedad, historia, gobierno y crecimiento**, no por tamaño.
Arquitectura medallion	Separa datos crudos (auditables y reprocesables) de datos limpios y de productos de negocio; cada capa tiene un dueño y una promesa clara.	Más tablas y pasos que mantener; duplica almacenamiento (barato en este volumen).
Lotes diarios y no *streaming*	La decisión de reposición se toma una vez al día o a la semana; latencia ≤ 24 h cumple el objetivo. Es más simple y barato de operar.	No sirve para alertas de stock intradía. Si se necesitara, Shopify ya entrega eventos y se podría pasar esa parte a Structured Streaming.
Webhooks de Shopify vía n8n	La organización ya usa n8n; evita desarrollar un conector propio.	n8n pasa a ser un punto único de falla para esa fuente: se mitiga con reintentos y con una conciliación diaria contra la API de Shopify.
Databricks como plataforma	Une Spark, orquestación, gobierno (Unity Catalog), SQL y MLflow en un solo lugar, lo que reduce piezas a operar con un equipo chico.	Costo por consumo y dependencia del proveedor; se acota con clusters que se apagan solos y datos en formatos abiertos (Delta/Parquet).
Calidad con reglas en Silver y cuarentena	Las reglas (SKU existente, unidades ≠ null, fecha válida, sin duplicados) frenan datos malos antes de Gold sin perder las filas rechazadas.	Las reglas hay que mantenerlas con las áreas dueñas del dato.
Unity Catalog y seudonimización	Cumple Ley 25.326 y limita costos/márgenes por rol. Linaje para auditoría.	Requiere definir roles y dueños de datos desde el inicio.
Power BI	Ya es la herramienta de las áreas usuarias; evita capacitación y se conecta al SQL Warehouse.	Licenciamiento aparte. Los modelos se mantienen en el Gold y no dentro de Power BI.
Seguridad y costo (estimación)	Cifrado en reposo/tránsito, acceso por rol, credenciales en un gestor de secretos y no en notebooks. Con este volumen se espera un costo mensual de plataforma bajo (orden de cientos de USD, **estimación sin cotizar**).	El costo real depende de la región, frecuencia de refresco de los tableros y uso de ML; hay que medirlo en el piloto.

Limitaciones generales del caso: todas las cifras son supuestas; no hay datos reales para validar la calidad de las fuentes; las metas de KPI son hipótesis que solo el piloto puede confirmar; y el modelo de reposición depende de que el stock registrado sea fiable (si el inventario físico difiere del sistema, la sugerencia hereda ese error).

---

6. Extensión opcional: caso de MLOps sobre la plataforma

Objetivo del modelo: pronosticar la **demanda semanal por categoría-local** (o por SKU-local en artículos de alta rotación) para las próximas 4 semanas, y usar ese pronóstico en `gold_sugerencia_reposicion`.

Features (tomadas de Gold): ventas de las últimas 1, 4 y 8 semanas, stock y días con stock cero (para no confundir "no vendió" con "no tenía"), atributos de producto (categoría, temporada), tipo de local, promoción vigente, semana del año y feriados, e indicadores macro (IPC, tipo de cambio).

Flujo de entrenamiento e inferencia 1. Una tabla de *features* se construye desde Gold con fecha de corte para evitar fuga de información. 2. Entrenamiento con partición **temporal** (entrenar en el pasado, validar en las últimas semanas), no aleatoria. 3. Se comparan al menos un *baseline* simple (promedio móvil) y un modelo de *gradient boosting* (p. ej. LightGBM). 4. Inferencia **batch semanal** como un paso más del pipeline; el resultado se escribe en `fact_reposicion` y el equipo de compras lo aprueba o ajusta (`unidades_aprobadas`).

Seguimiento de experimentos: MLflow registra parámetros, métricas (WAPE, sesgo), artefactos y versión del dataset de cada corrida. El mejor modelo se registra en el **Model Registry** con alias (`challenger`/`champion`) y solo se promueve si supera al baseline en la validación temporal.

Monitoreo: error del pronóstico contra ventas reales de cada semana (WAPE y sesgo), *drift* de las features principales, frescura y completitud de la tabla de features, y alerta si el error supera un umbral durante varias semanas seguidas (disparador de reentrenamiento). Las decisiones aprobadas por humanos también se comparan contra la sugerencia para medir si el modelo realmente aporta.

---

7. Fuentes y referencias

- Material de la cursada *Herramientas de Procesamiento para Grandes Volúmenes de Datos* (ITBA): clases de Databricks/medallion, NoSQL, MLOps, MLflow y observabilidad.
- Documentación de Databricks: Delta Lake, Auto Loader, Lakeflow Pipelines, Unity Catalog. https://docs.databricks.com
- Documentación de MLflow. https://mlflow.org/docs/latest/
- Ley 25.326 de Protección de los Datos Personales (Argentina).
- Kimball, R. y Ross, M., *The Data Warehouse Toolkit* (modelado dimensional).